

# Pretrained Computer Vision Models, Optimizers, Loss Functions, & Evaluation Metrics

## Comprehensive Technical Guide & Mathematical Foundations

Author: Gemini Notebook — Target Frameworks: PyTorch, TensorFlow, Hugging Face, Ultralytics

## 1 Introduction & Ecosystem Architecture

Modern Computer Vision (CV) relies heavily on **Transfer Learning**, leveraging pretrained backbone neural networks trained on massive datasets such as ImageNet-1k, ImageNet-22k, or COCO. Rather than training deep feature extractors from scratch, practitioners freeze or fine-tune feature-extraction layers and replace task-specific classification or regression heads.

### Core Frameworks vs. High-Level Model Repositories

- **Core Engines (Level 1):** PyTorch and TensorFlow serve as low-level tensor computation engines handling automatic differentiation (`autograd`), GPU CUDA execution, and custom layer construction.
- **High-Level Repositories (Level 2):**
  - `timm` (*PyTorch Image Models*): The definitive library for state-of-the-art vision backbones (ResNet, EfficientNet, ConvNeXt, Swin).
  - `torchvision`: Official PyTorch CV domain package containing standard datasets, transforms, and validated vision architectures.
  - `ultralytics`: Highly optimized framework specialized for real-time object detection, segmentation, and pose estimation (YOLOv8–v11).
  - `transformers` (*Hugging Face*): The standard hub for Transformer-based vision (ViT, Swin) and Multimodal architectures (CLIP, BLIP).
  - `tf.keras.applications`: TensorFlow's native repository for pretrained models.

## 2 Taxonomy of Pretrained Computer Vision Models

### 2.1 Image Classification Models

- **ResNet (e.g., ResNet50, ResNet101):** Introduces *Residual Skip Connections* ( $y = \mathcal{F}(x) + x$ ) to solve the vanishing gradient problem in deep networks. Serves as the universal baseline architecture.
- **EfficientNet (V1 / V2):** Uses compound scaling (jointly scaling depth, width, and image resolution). EfficientNetV2 replaces early depthwise separable convolutions with *Fused-MBConv* layers, significantly boosting training speed and parameter efficiency.
- **MobileNet (V3):** Employs Depthwise Separable Convolutions, Inverted Residuals, and Linear Bottlenecks designed specifically for resource-constrained edge devices (iOS, Android, IoT).
- **Vision Transformer (ViT):** Applies Self-Attention mechanisms directly to flattened non-overlapping image patches ( $16 \times 16$ ). Achieves superior top-1 accuracy on large datasets but lacks inductive bias (requiring heavier pretraining or strong augmentation).

### 2.2 Object Detection & Instance Segmentation Models

- **YOLO Family (YOLOv8, YOLOv9, YOLOv10, YOLOv11):** One-stage real-time detectors predicting bounding box coordinates and class probabilities in a single forward pass. Optimized for ultra-low latency inference.
- **Faster R-CNN:** Two-stage detector utilizing a Region Proposal Network (RPN) followed by RoI Pooling and classification/regression heads. Provides higher localization accuracy at the expense of speed.

- **Mask R-CNN:** Extends Faster R-CNN by adding a parallel Fully Convolutional Network (FCN) branch for pixel-level Instance Segmentation.
- **U-Net:** Symmetric Encoder-Decoder architecture with skip connections between corresponding resolutions. The industry standard for Semantic Segmentation in medical imaging (MRI, CT, X-ray).

### 3 Optimization Algorithms (Optimizers)

Optimizers update network weights  $\theta$  by computing gradients of the objective loss function  $\mathcal{L}(\theta)$  with respect to model parameters.

#### 3.1 Stochastic Gradient Descent (SGD) with Momentum

Accumulates past velocity vectors to accelerate descent in consistent directions and dampen oscillations in steep ravines:

$$\mathbf{v}_t = \gamma \mathbf{v}_{t-1} + \eta \nabla_{\theta} \mathcal{L}(\theta_{t-1}) \quad (1)$$

$$\theta_t = \theta_{t-1} - \mathbf{v}_t \quad (2)$$

where  $\eta$  is the learning rate and  $\gamma \in [0, 1)$  is the momentum coefficient (typically  $\gamma = 0.9$ ). Recommended for training classic ConvNets (ResNet) from scratch due to superior generalization bounds.

#### 3.2 Adam (Adaptive Moment Estimation)

Combines the benefits of Momentum (first moment  $m_t$ ) and RMSprop (second uncentered moment  $v_t$ ) to compute adaptive learning rates for each parameter:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (3)$$

Bias-corrected estimate computation:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (4)$$

Parameter update rule:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (5)$$

where default hyperparameters are  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ , and  $\epsilon = 10^{-8}$ .

#### 3.3 AdamW (Adam with Decoupled Weight Decay)

Fixes  $L_2$  regularization in original Adam where weight decay was incorrectly coupled with adaptive gradients. Decouples weight decay ( $\lambda$ ) directly into the parameter update step:

$$\theta_t = (1 - \eta\lambda)\theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (6)$$

**Essential Rule:** AdamW is mandatory when fine-tuning Vision Transformers (ViT, Swin) and modern ConvNets (ConvNeXt).

## 4 Mathematical Formulations of Loss Functions

#### 4.1 Binary Cross-Entropy (BCE) Loss

Used for binary classification or multi-label classification tasks (paired with Sigmoid activation):

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (7)$$

where  $y_i \in \{0, 1\}$  and  $p_i \in (0, 1)$  is the predicted probability.

## 4.2 Categorical Cross-Entropy (CCE) Loss

Used for single-label multi-class classification (paired with Softmax activation):

$$\mathcal{L}_{\text{CCE}} = - \sum_{c=1}^C y_c \log(p_c) \quad (8)$$

where  $C$  is the total number of classes and  $y_c$  is a one-hot encoded ground truth vector.

## 4.3 Focal Loss (Addressing Extreme Class Imbalance)

Modulates Cross-Entropy loss by down-weighting well-classified easy examples ( $(1 - p_t) \rightarrow 0$ ) and focusing gradients on hard, minority samples:

$$\mathcal{L}_{\text{Focal}}(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (9)$$

where  $p_t$  is the model's estimated probability for the ground truth class,  $\gamma \geq 0$  is the focusing parameter (typically  $\gamma = 2.0$ ), and  $\alpha_t \in [0, 1]$  balances class frequencies.

## 4.4 Smooth $L_1$ Loss (Huber Loss)

Combines  $L_2$  loss for small errors (smooth gradients) and  $L_1$  loss for large errors (robustness to outliers) in bounding box regression:

$$\mathcal{L}_{\text{Smooth } L_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases} \quad (10)$$

## 4.5 Complete Intersection over Union (CIoU) Loss

Optimizes bounding box regression in modern YOLO models by evaluating overlap area, central point distance, and aspect ratio consistency:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(b, b^{\text{gt}})}{c^2} + \alpha v \quad (11)$$

where  $\rho(b, b^{\text{gt}})$  is the Euclidean distance between box center points,  $c$  is the diagonal length of the smallest enclosing box,  $v = \frac{4}{\pi^2} \left( \arctan \frac{w^{\text{gt}}}{h^{\text{gt}}} - \arctan \frac{w}{h} \right)^2$ , and  $\alpha = \frac{v}{(1 - \text{IoU}) + v}$ .

## 4.6 Dice Loss

Measures region overlap in Semantic Segmentation, mitigating severe background-foreground pixel imbalance:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2|A \cap B| + \epsilon}{|A| + |B| + \epsilon} \quad (12)$$

# 5 Evaluation Metrics

## 5.1 Classification Metrics

- **Top-1 / Top-5 Accuracy:** Percentage of samples where the true class matches the top-1 or top-5 highest probability predictions.
- **Precision, Recall, and  $F_1$ -Score:**

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (13)$$

## 5.2 Detection & Segmentation Metrics

- **Intersection over Union (IoU):** Ratio of overlap area over union area between predicted bounding box  $B_{\text{pred}}$  and ground truth box  $B_{\text{gt}}$ :

$$\text{IoU} = \frac{|B_{\text{pred}} \cap B_{\text{gt}}|}{|B_{\text{pred}} \cup B_{\text{gt}}|} \quad (14)$$

- **mAP@50 & mAP@50:95:** Mean Average Precision evaluated at IoU threshold 0.50, and averaged across IoU thresholds from 0.50 to 0.95 (step 0.05).
- **mIoU (Mean IoU):** Average pixel-wise IoU calculated across all semantic classes.

## 6 Recommended Component Combinations (Stacks)

The table below outlines optimal combinations of Pretrained Backbones, Optimizers, Loss Functions, and Primary Metrics widely adopted in industrial and competitive AI applications:

| Task / Domain                  | Pretrained Models                     | Optimizer                  | Loss Function                                | Primary Metrics                         |
|--------------------------------|---------------------------------------|----------------------------|----------------------------------------------|-----------------------------------------|
| Multi-Class Classification     | ResNet50, EfficientNetV2, MobileNetV3 | SGD (m=0.9) / Adam         | Categorical Cross-Entropy                    | Top-1 / Top-5 Accuracy                  |
| Multi-Label Classification     | ResNet50, ViT-Base                    | AdamW                      | Binary Cross-Entropy (BCE)                   | Macro $F_1$ , Hamming Loss              |
| Imbalanced Data Classification | EfficientNetV2-S, ResNet101           | Adam / AdamW               | Focal Loss ( $\gamma = 2.0, \alpha = 0.25$ ) | Precision, Recall, Macro $F_1$ , PR-AUC |
| Vision Transformer Fine-Tuning | ViT-B/16, Swin-Base                   | AdamW ( $\lambda = 0.05$ ) | Cross-Entropy + Label Smoothing              | Top-1 Accuracy, $F_1$ -Score            |
| Real-Time Object Detection     | YOLOv8 / YOLOv11                      | SGD / AdamW                | CIoU + DFL Box Loss + Focal Class Loss       | mAP@50, mAP@50:95, FPS                  |
| Two-Stage Object Detection     | Faster R-CNN (ResNet50 FPN)           | SGD (m=0.9)                | RPN Loss + Smooth $L_1$ + Cross-Entropy      | mAP@50:95                               |
| Medical Image Segmentation     | U-Net (ResNet34 Backbone)             | Adam / RMSprop             | Combined BCE + Dice Loss                     | Dice Coefficient, mIoU                  |

## 7 Golden Rules of System Integration

- Task  $\leftrightarrow$  Loss Function (Strict Coupling):** The loss function must strictly match the output activation and task formulation. Multi-class single-label problems require Softmax + Cross-Entropy; multi-label problems require Sigmoid + BCE. Mismatched losses cause divergence or runtime crashes.
- Architecture  $\leftrightarrow$  Optimizer (Best Practice):** Standard CNNs (ResNet, Faster R-CNN) perform well with SGD + Momentum. Transformer architectures (ViT, Swin) require AdamW to properly manage weight decay across self-attention projections.
- Task  $\leftrightarrow$  Metrics (Evaluation Only):** Metrics monitor model performance without altering network weights. Multiple metrics can be tracked simultaneously during training without impacting model optimization.